

Mise en place d'une sauvegarde

**BTS SIO (Services Informatiques aux
Organisations)**

**Option Solutions d'Infrastructure, Systèmes et
Réseaux (SISR)**

Zoha ASHFAQ

Sommaire

Sommaire	2
Le tableau	3
Création d'un snapshot de la machine virtuelle	7
Ajout et montage d'un disque dédié aux sauvegardes	7
Étape 1 — État des lieux avant ajout	7
Étape 2 — Ajouter un disque dans VMware Workstation Pro.....	8
Étape 3 — Identification du nouveau disque.....	8
Étape 4 — Partitionner le disque.....	9
Étape 5 — Formater la partition	9
Étape 6 — Montage permanent avec l'UUID.....	9
Sauvegarde manuelle de la base GLPI avec mysqldump	11
Étape 1 — Préparer le dossier de sauvegarde	11
Étape 2 — Lancement de la sauvegarde manuelle	11
Vérifications finales	17

	BTS SIO		
	Services Informatiques aux Organisations		
	Option	SISR	
	Session	2026	

Le tableau

	Activité professionnelle N°5	
--	------------------------------	--

NATURE DE L'ACTIVITE	Déploiement d'une stratégie de sauvegarde de la base MariaDB du serveur GLPI de l'entreprise M2L : mise en œuvre d'un disque secondaire dédié, configuration de mysqldump avec rotation automatique via cron, et validation de la procédure de restauration.
Contexte	La Maison des Ligues de Lorraine (M2L) dispose d'un serveur GLPI hébergé sur Debian 12, utilisé pour centraliser la gestion du parc informatique ainsi que le suivi des tickets d'assistance pour les 35 ligues sportives. La base de données MariaDB contient des informations essentielles telles que l'inventaire du matériel, les tickets d'incident, les contrats et l'historique des interventions. Actuellement, aucune procédure de sauvegarde n'est mise en place. En cas de panne du disque ou de corruption de la base de données, l'ensemble de ces informations pourrait être perdu définitivement.
Objectifs	Mettre en place une sauvegarde quotidienne automatisée de la base de données Gestionnaire Libre de Parc Informatique (GLPI) avec un Objectif de Point de Reprise (Recovery Point Objective – RPO) de 24 heures. Assurer une rotation des sauvegardes sur 7 jours. Documenter la procédure de restauration. Vérifier le bon fonctionnement par un test complet de restauration.
Lieu de réalisation	Ecole, Aurlom BTS +

SOLUTIONS ENVISAGEABLES

- Sauvegarde manuelle par export phpMyAdmin : accessible sans ligne de commande mais non automatisée, non compressée, et inutilisable en cas de panne du serveur web.
- Script Bash utilisant mysqldump, cron et gzip : solution native Linux, entièrement automatisée, compressée et avec rotation. Solution retenue.

- Outil de sauvegarde automatisé Bacula : très complet mais complexe à déployer pour un besoin limité sur une seule base SQL.
- Réplication de base de données MariaDB (Master/Slave) : performante mais surdimensionnée pour l'infrastructure actuelle

DESCRIPTION DE LA SOLUTION RETENUE

Conditions initiales	- Machine virtuelle Debian 12 avec un unique disque de 60 Go partitionné sous LVM. - Base MariaDB glpi en production, contenant tickets, inventaire et comptes utilisateurs. - Pas de politique de sauvegarde définie : toute panne disque entraînerait une perte totale et irréversible. - Accès administrateur (root) disponible via SSH depuis VMware Workstation Pro.
Conditions finales	- Disque virtuel secondaire de 20 Go ajouté dans VMware Workstation Pro, formaté en ext4, monté de façon permanente sur /backups grâce à son UUID dans /etc/fstab. - Script Bash /usr/local/bin/backup-glpi.sh en place : dump compressé avec gzip, rotation automatique des fichiers de plus de 7 jours. - Sauvegarde planifiée chaque nuit à 2h00 via crontab, logs dans /var/log/backup-glpi.log. - Test de restauration concluant : simulation d'une perte de la table glpi_tickets, récupération intégrale des données depuis le dernier dump compressé, vérification des 5 tickets retrouvés.
Outils utilisés	- VMware Workstation Pro 17 (virtualisation du serveur GLPI sous Debian 12) - mysqldump (outil natif MariaDB pour l'export de la base de données en SQL) - Bash + cron (automatisation et planification des sauvegardes) - gzip (compression des fichiers .sql pour réduire l'espace occupé) - nano (édition des fichiers de configuration)

CONDITIONS DE REALISATION

Matériels	- VMware Workstation Pro 17 - VM Debian 12 : disque système 60 Go + disque sauvegarde 20 Go - Poste administrateur sous Windows 11 avec accès SSH
Logiciels	- Debian 12 (système d'exploitation du serveur GLPI) - MariaDB 10.11 - GLPI (application de gestion du parc informatique) - mysqldump, gzip, cron (chaîne complète de sauvegarde automatisée) - fdisk, mkfs.ext4, blkid (gestion du disque de sauvegarde dédié).

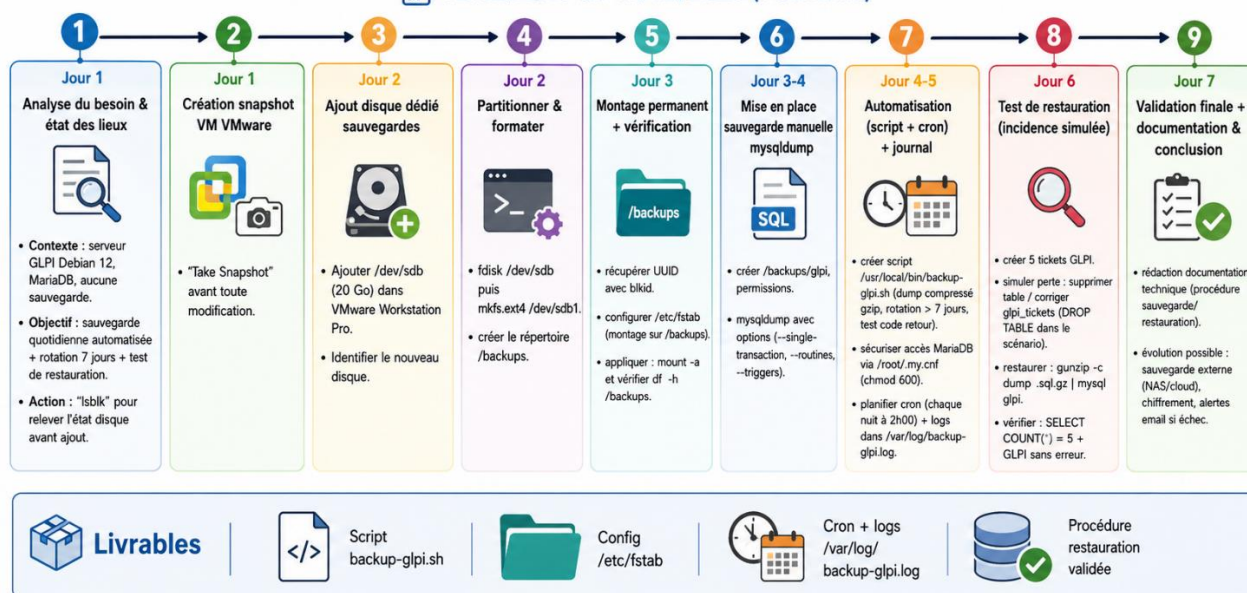
Durée	1 semaine (du 12/02/2026 au 19/02/2026) : : analyse du besoin, ajout du disque virtuel, configuration de la sauvegarde, tests et validation de la restauration et documentation.
Contraintes	- La sauvegarde ne doit pas impacter les performances du serveur GLPI en journée. - L'authentification MariaDB doit être automatisée sans stocker le mot de passe en clair dans le script. - La rotation des sauvegardes doit être automatisée pour éviter la saturation du disque /backups. - Le montage du disque /backups doit être persistant après redémarrage sans rendre le serveur inaccessible. - La procédure de restauration doit être documentée et testable par n'importe quel administrateur.

COMPETENCES MISES EN OEUVRE POUR CETTE ACTIVITE PROFESSIONNELLE

Répondre aux incidents et aux demandes d'assistance et d'évolution	Diagnostic du risque de perte de données sur GLPI (absence de sauvegarde) puis résolution d'un incident simulé : corruption/suppression de la table glpi_tickets((DROP TABLE glpi_tickets). Restauration complète de la base depuis le dump MariaDB compressé, puis validation fonctionnelle (retour des tickets dans GLPI sans erreur). Mise en place d'une stratégie de sauvegarde automatisée et testée pour prévenir la récurrence.
Mettre à disposition un service informatique :	Je mets à disposition un service de sauvegarde pour la base GLPI : j'ajoute et configure un disque dédié monté sur /backups avec un montage permanent via UUID dans /etc/fstab, je sécurise l'authentification MariaDB avec /root/.my.cnf, je mets en place un script de sauvegarde backup-glpi.sh utilisant mysqldump et gzip avec une rotation automatique des sauvegardes sur 7 jours, et j'active la planification quotidienne via cron à 2h00. Je configure la journalisation dans /var/log/backup-glpi.log, et je valide le fonctionnement par un test de restauration (simulation de suppression de la table glpi_tickets, restauration depuis le dump compressé, puis vérification des 5 tickets et du bon fonctionnement de GLPI).
Gérer le patrimoine informatique :	Je gère le patrimoine informatique en mettant en place et en assurant la disponibilité des données de la plateforme GLPI : je crée un snapshot avant modification, j'ajoute une extension de stockage (disque /dev/sdb partitionné et formaté en ext4), je configure un montage permanent sur /backups grâce à l'UUID dans /etc/fstab, et je mets en place une politique de conservation avec une rotation sur 7 jours. Je valide la récupération des données par un test de restauration complet après suppression de la table glpi_tickets.
Travailler en mode projet :	Je structure mon intervention en mode projet : je découpe la mission en étapes (snapshot, ajout du disque, montage permanent, script et cron, tests, restauration), je respecte le planning prévu sur la semaine, je documente la procédure de sauvegarde et de restauration, et je valide la solution par un test complet de restauration après suppression de la table glpi_tickets.

Planning – Mission Sauvegarde (BTS SIO – SISR)

12/02/2026 au 19/02/2026 (1 semaine)



REALISATION

J'ai d'abord réalisé un snapshot de la machine virtuelle afin de pouvoir revenir à un état fonctionnel en cas de mauvaise manipulation. J'ai ensuite ajouté un disque virtuel dédié aux sauvegardes, que j'ai partitionné, formaté en ext4 puis monté de manière permanente sur /backups grâce à son UUID dans /etc/fstab.

J'ai ensuite créé un dossier spécifique pour les sauvegardes de GLPI, puis j'ai effectué une première sauvegarde manuelle de la base MariaDB avec mysqldump afin de vérifier le bon fonctionnement de la procédure. Après cette validation, j'ai sécurisé l'authentification MariaDB avec le fichier /root/.my.cnf, puis j'ai écrit un script Bash de sauvegarde automatisée utilisant mysqldump, gzip et une rotation des sauvegardes sur 7 jours.

Enfin, j'ai planifié l'exécution quotidienne du script avec cron à 2h00 du matin, puis j'ai testé la procédure de restauration en supprimant la table glpi_tickets, avant de restaurer la base depuis le dernier dump compressé et de vérifier que les 5 tickets étaient bien revenus dans GLPI.

CONCLUSION

Cette intervention m'a permis de concevoir et de déployer une solution de sauvegarde robuste pour le serveur GLPI de la M2L. L'approche retenue, basée sur mysqldump et cron, est économique, fiable et complètement automatisée, sans impact sur la disponibilité du service. Cela répond à toutes les contraintes : zéro interruption de service, sécurité de l'authentification, rotation automatique. J'ai consolidé des compétences fondamentales en administration système Linux : gestion des disques et partitions, scripts Bash, administration de bases de données MariaDB et planification de tâches avec cron. La fiabilité de la solution a été confirmée par un test : après la suppression volontaire de la table glpi_tickets a été restaurée en quelques secondes à partir du dump compressé.

EVOLUTION POSSIBLE

- Sauvegarde externe : transférer les archives vers un serveur NAS ou un stockage cloud pour protéger les données en cas de panne matérielle.

- Protection des données : chiffrer les fichiers SQL pour sécuriser les informations sensibles des utilisateurs de GLPI.
- Alertes automatiques : envoyer un e-mail au responsable informatique si une sauvegarde échoue, avec le journal d'exécution.
- Sauvegarde complète : inclure les fichiers de configuration et les pièces jointes de GLPI pour couvrir l'ensemble du système.

Création d'un snapshot de la machine virtuelle

Avant de réaliser une sauvegarde, je crée un snapshot de la machine virtuelle du serveur GLPI afin de pouvoir revenir rapidement à un état fonctionnel en cas de mauvaise manipulation.

Si une erreur venait à corrompre le système ou la base de données, ce snapshot permettra de restaurer l'état initial du serveur en quelques secondes. Dans VMware Workstation : clic droit sur la VM → *Snapshot* → *Take Snapshot*, nommez-le.



La création d'un snapshot constitue une bonne pratique en environnement virtualisé, car elle permet de sécuriser les manipulations et de revenir rapidement à un état stable avant toute modification importante du système.

Ajout et montage d'un disque dédié aux sauvegardes

Stocker les sauvegardes sur le même disque que les données de production est inutile en cas de panne disque. Nous allons ajouter un second disque virtuel au serveur GLPI et le monter sur /backups. Le disque principal du serveur est sous LVM : le disque de sauvegarde sera plus simple, en partition classique ext4.

Étape 1 — État des lieux avant ajout

Avant toute modification, je prends un instantané de la configuration disque actuelle. C'est un réflexe essentiel : toujours documenter l'état initial avant d'intervenir sur le stockage.

Commande : lsblk


```

root@srv-glpi:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   60G  0 disk
├─sda1                              8:1      0  487M  0 part /boot
├─sda2                              8:2      0    1K  0 part
└─sda5                              8:5      0  59,5G  0 part
    ├─deb12--master--vg-root        254:0    0  58,6G  0 lvm  /
    └─deb12--master--vg-swap_1      254:1    0   976M  0 lvm  [SWAP]
sr0                                  11:0     1   738M  0 rom
root@srv-glpi:~#

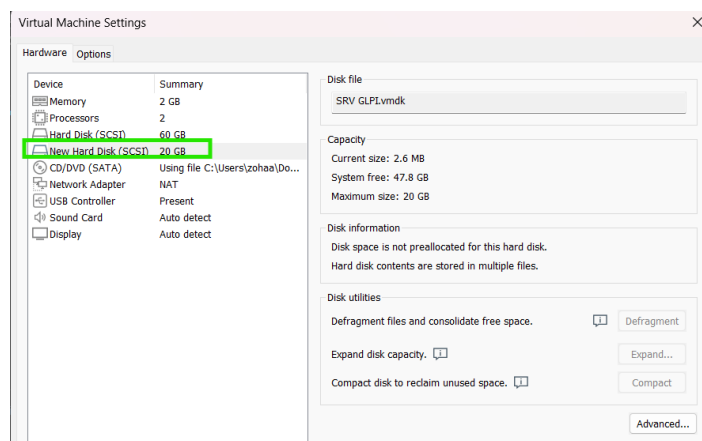
```

J'ai un seul disque /dev/sda de 60 Go avec le LVM contenant les volumes système. Je note bien cette sortie pour la comparer après l'ajout du disque.

Étape 2 — Ajouter un disque dans VMware Workstation Pro

J'éteigne proprement la machine virtuelle du serveur GLPI, puis dans VMware Workstation Pro :

Menu VM → Settings → Add → Hard Disk → SCSI → Create a new virtual disk



J'attribue une taille adaptée (par exemple 20 Go, suffisant pour plusieurs semaines de dumps GLPI compressés). Je valide et redémarre la machine virtuelle.

Étape 3 — Identification du nouveau disque

Après le redémarrage, je relance lsblk pour comparer avec l'état initial :

```
root@srv-glpi:~# lsblk
```

Le nouveau disque /dev/sda de 20 Go apparaît, vierge, sans aucune partition.

```

admin@srv-glpi:~$ su -
Mot de passe :
root@srv-glpi:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   20G  0 disk
sdb                                  8:16     0   60G  0 disk
├─sdb1                              8:17     0  487M  0 part /boot
├─sdb2                              8:18     0    1K  0 part
└─sdb5                              8:21     0  59,5G  0 part
    ├─deb12--master--vg-root        254:0    0  58,6G  0 lvm  /
    └─deb12--master--vg-swap_1      254:1    0   976M  0 lvm  [SWAP]
sr0                                  11:0     1   738M  0 rom
root@srv-glpi:~#

```


Étape 4 — Partitionner le disque

Je crée une partition unique occupant tout le disque avec fdisk. J'assure de bien cibler /dev/sda (le disque sans LVM) :

Commande : fdisk /dev/sda

Puis une vérification avec la commande : lsblk /dev/sda

```
root@srv-glpi:~# fdisk /dev/sda

Bienvenue dans fdisk (util-linux 2.38.1).
Les modifications resteront en mémoire jusqu'à écriture.
Soyez prudent avant d'utiliser la commande d'écriture.

Le périphérique ne contient pas de table de partitions reconnue.
Created a new DOS (MBR) disklabel with disk identifier 0xalb89719.

Commande (m pour l'aide) : n
Type de partition
  p primaire (0 primaire, 0 étendue, 4 libre)
  e étendue (conteneur pour partitions logiques)
Sélectionnez (p par défaut) : p
Numéro de partition (1-4, 1 par défaut) : 1
Premier secteur (2048-41943039, 2048 par défaut) :
Dernier secteur, +/-secteurs ou +/-taille{K,M,G,T,P} (2048-41943039, 41943039 par défaut) :

Une nouvelle partition 1 de type « Linux » et de taille 20 GiB a été créée.

Commande (m pour l'aide) : w
La table de partitions a été altérée.
Appel d'ioctl() pour relire la table de partitions.
Synchronisation des disques.

root@srv-glpi:~# lsblk /dev/sda
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda           8:0    0   20G  0 disk
└─sda1       8:1    0   20G  0 part
```

Étape 5 — Formater la partition

J'utilise la commande mkfs.ext4 /dev/sda1 pour formater la partition, puis la commande mkdir /backups pour créer le répertoire qui servira de point de montage.

```
root@srv-glpi:~# mkfs.ext4 /dev/sda1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 5242624 4k blocks and 1310720 inodes
Filesystem UUID: 2aaa0478-5f94-4ae0-86a4-ef3d6cb6d38a
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000

Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

root@srv-glpi:~# mkdir /backups
```

Étape 6 — Montage permanent avec l'UUID

Plutôt que d'utiliser /dev/sda1 dans /etc/fstab (car ce nom peut changer selon l'ordre de détection des disques), je récupère l'UUID (identifiant universel unique) de la partition avec la commande blkid /dev/sda1. Cet identifiant est propre à chaque partition et reste toujours le même, ce qui permet de monter la partition de manière fiable au démarrage.

```
root@srv-glpi:~# blkid /dev/sda1
/dev/sda1: UUID="2aaa0478-5f94-4ae0-86a4-ef3d6cb6d38a" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="a1b89719-01"
```

Ensuite, j'édite le fichier /etc/fstab, qui permet de définir les systèmes de fichiers à monter automatiquement au démarrage du système :

J'ajoute à la fin du fichier la ligne suivante (en remplaçant l'UUID par celui obtenu avec blkid) :

UUID=2aaa0478-5f94-4ae0-86a4-ef3d6cb6d38a /backups ext4 defaults 0 2

```
admin1@srv-glpi: ~
GNU nano 7.2 /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# systemd generates mount units based on this file, see systemd.mount(5).
# Please run 'systemctl daemon-reload' after making changes here.
#
# <file system> <mount point> <type> <options> <dump> <pass>
/dev/mapper/debl2--master--vg-root / ext4 errors=remount-ro 0 1
# /boot was on /dev/sda1 during installation
UUID=d06e1780-4fb5-4ed0-b064-fd926fd321ad /boot ext2 defaults 0 2
/dev/mapper/debl2--master--vg-swap_1 none swap sw 0 0
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0
UUID=2aaa0478-5f94-4ae0-86a4-ef3d6cb6d38a /backups ext4 defaults 0 2
```

Cette ligne indique au système de monter automatiquement la partition au démarrage dans le répertoire /backups.

Pour appliquer immédiatement la configuration et monter la partition, j'exécute les commandes suivantes : **Commande : mount -a** puis **Commande : systemctl daemon-reload**

Afin de vérifier que la partition est correctement montée, j'utilise les commandes suivantes :

Commande : mount | grep sda1 puis **Commande : df -h /backups**

Ces commandes permettent de confirmer que la partition est bien montée sur le répertoire /backups et d'afficher l'espace disque disponible.

```
root@srv-glpi:~# mount -a
montage : (astuce) votre fstab a été modifié mais systemd utilise encore
l'ancienne version ; utilisez « systemctl daemon-reload » pour recharger.
root@srv-glpi:~# systemctl daemon-reload
root@srv-glpi:~# mount | grep sdb1
/dev/sdb1 on /boot type ext2 (rw,relatime)
root@srv-glpi:~# mount | grep sda1
/dev/sda1 on /backups type ext4 (rw,relatime)
root@srv-glpi:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   20G  0 disk
├─sda1                              8:1      0   20G  0 part /backups
sdb                                  8:16     0    60G  0 disk
├─sdb1                              8:17     0  487M  0 part /boot
├─sdb2                              8:18     0     1K  0 part
└─sdb5                              8:21     0  59,5G  0 part
    └─debl2--master--vg-root        254:0     0  58,6G  0 lvm /
        └─debl2--master--vg-swap_1 254:1     0  976M  0 lvm [SWAP]
sr0                                 11:0     1  738M  0 rom
root@srv-glpi:~# mount | grep sda1
/dev/sda1 on /backups type ext4 (rw,relatime)
root@srv-glpi:~# df -h /backups
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/sda1          20G      24K   19G   1% /backups
```

Le disque de 20 Go est maintenant monté sur /backups et prêt à être utilisé pour stocker les sauvegardes.

Bonne pratique : disque dédié

Dans un environnement de production, il est recommandé d'utiliser un disque de sauvegarde physiquement distinct du disque de production. Dans notre environnement virtualisé, cela correspond à deux fichiers .vmdk différents représentant deux disques virtuels. En entreprise, les sauvegardes sont souvent stockées sur un serveur de stockage dédié ou un NAS. L'objectif est d'éviter qu'une panne du disque principal entraîne également la perte des sauvegardes, garantissant ainsi une meilleure sécurité des données.

Sauvegarde manuelle de la base GLPI avec mysqldump

Je vais réaliser pas à pas une sauvegarde manuelle de la base de données GLPI. L'objectif est de comprendre le fonctionnement de mysqldump avant de l'automatiser.

Étape 1 — Préparer le dossier de sauvegarde

On crée un dossier dédié aux sauvegardes de la base “/backups/glpi”. Il est important de séparer les sauvegardes des données de production et de restreindre les droits d'accès (le dump SQL contient toutes les données de la base, y compris les mots de passe hashés des utilisateurs GLPI).

Commande: `mkdir -p /backups/glpi`

Commande: `chmod 700 /backups/glpi`

```
root@srv-glpi:~# mkdir -p /backups/glpi
root@srv-glpi:~# chmod 700 /backups/glpi
```

Étape 2 — Lancement de la sauvegarde manuelle

La commande mysqldump se connecte à MariaDB, lit la structure et le contenu de la base, puis génère un fichier SQL contenant toutes les instructions nécessaires pour la recréer de zéro.

Commande : `mysqldump -u glpi -p --single-transaction --routines --triggers glpi > /backups/glpi/glpi_$(date +%Y%m%d-%H%M%S).sql`

Pas besoin d'arrêter GLPI !

Grâce à l'option `--single-transaction`, l'export se fait à chaud : MariaDB fournit un instantané cohérent de la base sans verrouiller les tables. Les utilisateurs de GLPI continuent de travailler normalement pendant la sauvegarde. C'est toute la différence avec une sauvegarde à froid qui imposerait un `systemctl stop apache2` puis un `systemctl stop mariadb` avant la copie des fichiers.

Je vérifie le résultat

Après l'exécution, je vérifie que le fichier a bien été créé et qu'il a une taille cohérente (`ls -lh`). Un dump GLPI fonctionnel pèse au minimum quelques Mo. **Commande :** `ls -lh /backups/glpi/`

Je peux aussi vérifier les premières et dernières lignes du fichier pour s'assurer qu'il est complet :

Commande : `head -20 /backups/glpi/glpi_20260217-11165.sql`

```

root@srv-glp1:~# mysqldump -u glpi -p --single-transaction --routines --triggers
glpi > /backups/glpi/glpi $(date +%Y%m%d-%H%M%S).sql
Enter password:
root@srv-glp1:~# ls -lh /backups/glpi/
total 2,0M
-rw-r--r-- 1 root root 2,0M 17 févr. 11:16 glpi 20260217-111651.sql
root@srv-glp1:~# head -20 /backups/glpi/glpi 20260216-143022.sql
head: impossible d'ouvrir '/backups/glpi/glpi_20260216-143022.sql' en lecture: A
ucun fichier ou dossier de ce type
root@srv-glp1:~# ls /backups/
glpi lost+found
root@srv-glp1:~# ls /backups/glpi/
glpi 20260217-111651.sql
root@srv-glp1:~# head -20 /backups/glpi/glpi 20260217-111651.sql
/*!999999\~ enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.11.14-MariaDB, for debian-linux-gnu (x86_64)
--
-- Host: localhost      Database: glpi
--
-- Server version      10.11.14-MariaDB-0+deb12u2

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0
*/;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO AUTO VALUE ON ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Table structure for table `glpi agents`
root@srv-glp1:~# tail -5 /backups/glpi/glpi 20260217-111651.sql
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;

-- Dump completed on 2026-02-17 11:16:59

```

Automatiser la sauvegarde avec cron

Une sauvegarde manuelle c'est bien pour comprendre, mais en production personne ne lance mysqldump à la main tous les jours à 2h du matin. *Je vais automatiser le processus en trois étapes : sécuriser l'authentification, écrire un script de sauvegarde, puis le planifier avec cron.*

L'automatisation repose sur trois éléments complémentaires : la sécurisation de l'authentification MariaDB via /root/.my.cnf, un script Bash dédié gérant le dump, la compression et la rotation, et une entrée cron planifiant l'exécution quotidienne.

gzip est-il installé ?

Avant d'utiliser un outil, je vérifie toujours qu'il est disponible. gzip fait partie des paquets essentiels de Debian et est présent par défaut sur toute installation, mais c'est un bon réflexe à prendre systématiquement :

```
root@srv-glp1:~# gzip --version
gzip 1.12 Copyright (C) 2022 Free Software Foundation, Inc.
```

Si la commande retourne une erreur, on peut l'installer avec apt install gzip.

Étape 1 — Sécurisation de l'authentification avec .my.cnf

Pour que mysqldump puisse s'exécuter sans intervention humaine, il doit disposer des identifiants MariaDB. Plutôt que de mettre le mot de passe en clair dans un script, on utilise le fichier ~/.my.cnf qui est lu automatiquement par tous les outils MariaDB/MySQL.

Commande : nano /root/.my.cnf

Contenu du fichier : [client] user=glpi password=VOTRE_MOT_DE_PASSE

Je verrouille les permissions pour que seul root puisse le lire : Commande : chmod 600 /root/.my.cnf

Je peux maintenant tester que mysqldump fonctionne sans demander le mot de passe :

Commande : `mysqldump --single-transaction glpi > /dev/null`

`echo $?`

```
root@srv-glpi:~# gzip --version
gzip 1.12
Copyright (C) 2018 Free Software Foundation, Inc.
Copyright (C) 1993 Jean-loup Gailly.
This is free software. You may redistribute copies of it under the terms of
the GNU General Public License <https://www.gnu.org/licenses/gpl.html>.
There is NO WARRANTY, to the extent permitted by law.

Written by Jean-loup Gailly.
root@srv-glpi:~# nano /root/.my.cnf
root@srv-glpi:~# chmod 600 /root/.my.cnf
root@srv-glpi:~# mysqldump --single-transaction glpi > /dev/null
root@srv-glpi:~# echo $?
0
```

Le code retour 0 confirme que l'export s'est bien déroulé.

Étape 2 — Écrire le script de sauvegarde

Je crée un script Bash dédié qui réalise le dump, compresse le fichier et supprime les sauvegardes de plus de 7 jours (rotation automatique).

Commande : `nano /usr/local/bin/backup-glpi.sh`

Le script centralise l'ensemble de la logique : vérification préalable de l'authentification, génération du dump compressé à la volée via un pipe (sans fichier intermédiaire non compressé), contrôle du code retour et rotation automatique des anciennes sauvegardes.

Contenu du script :

```
admin1@srv-glpi: ~
GNU nano 7.2
#!/bin/bash
# =====
# Script de sauvegarde de la base GLPI
# M2L - Maison des Lignes de Lorraine
# =====

# Variables
BACKUP_DIR="/backups/glpi"
DATE=$(date +%Y%m%d-%H%M%S)
FILENAME="glpi_${DATE}.sql.gz"
RETENTION=7 # Nombre de jours de conservation

# Vérification du fichier d'authentification MariaDB
# (créé à l'étape 1 - sécurité en cas de suppression accidentelle)
if [ ! -f /root/.my.cnf ]; then
    echo "[ERREUR] Fichier /root/.my.cnf introuvable !" >&2
    echo "      mysqldump ne pourra pas s'authentifier." >&2
    exit 1
fi

# Création du dossier si nécessaire
mkdir -p "$BACKUP_DIR"

# Dump de la base + compression à la volée
mysqldump --single-transaction --routines --triggers glpi \
| gzip > "${BACKUP_DIR}/${FILENAME}"

# Vérification du succès
if [ $? -eq 0 ]; then
    echo "[OK] Sauvegarde réussie : ${FILENAME}"
else
    echo "[ERREUR] Échec de la sauvegarde !" >&2
    exit 1
fi

# Rotation : suppression des fichiers de plus de N jours
find "$BACKUP_DIR" -name "glpi_*.sql.gz" -mtime +${RETENTION} -delete

echo "[OK] Rotation effectuée (conservation : ${RETENTION} jours)"
```

Pourquoi compresser avec gzip ?

Un dump SQL est un fichier texte très répétitif (instructions INSERT en masse). La compression avec gzip réduit typiquement la taille de 80 à 90 %. Notre dump de 12 Mo passe à environ 2 Mo. Sur 7 jours de rétention, cela représente 14 Mo au lieu de 84 Mo. L'opérateur pipe | permet de compresser à la volée sans créer de fichier intermédiaire non compressé.

Je rends le script exécutable avec la commande : `chmod 700 /usr/local/bin/backup-glpi.sh`

Je teste le script manuellement :

Commande : `/usr/local/bin/backup-glpi.sh`

root@srv-glpi:~# ls -lh /backups/glpi/

```
root@srv-glpi:~# nano /usr/local/bin/backup-glpi.sh
root@srv-glpi:~# chmod 700 /usr/local/bin/backup-glpi.sh
root@srv-glpi:~# /usr/local/bin/backup-glpi.sh
[OK] Sauvegarde réussie : glpi_20260217-114932.sql.gz
[OK] Rotation effectuée (conservation : 7 jours)
root@srv-glpi:~# ls -lh /backups/glpi/
total 2,2M
-rw-r--r-- 1 root root 2,0M 17 févr. 11:16 glpi_20260217-111651.sql
-rw-r--r-- 1 root root 223K 17 févr. 11:49 glpi_20260217-114932.sql.gz
```

Étape 3 — Planification avec cron

Le démon cron exécute des commandes à intervalles réguliers selon une planification définie. On édite la crontab de root avec la commande : `crontab -e`

Ensuite, je sélectionne le choix 1 pour utiliser nano comme éditeur.

J'ajoute la ligne suivante pour lancer la sauvegarde chaque nuit à 2h du matin :

```
root@srv-glpi:~# crontab -e
no crontab for root - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny

Choose 1-3 [1]: 1
crontab: installing new crontab
```

```
admin1@srv-glpi: ~
GNU nano 7.2 /tm
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
# Sauvegarde quotidienne de la base GLPI - M2L
0 2 * * * /usr/local/bin/backup-glpi.sh >> /var/log/backup-glpi.log 2>&1
```


La redirection `>> /var/log/backup-glpi.log 2>&1` enregistre la sortie standard et les erreurs dans un fichier de log consultable en cas de problème.

⚠ Vérification importante

Après la première nuit, pensez à vérifier que la sauvegarde automatique a bien fonctionné : consultez le fichier de log avec `cat /var/log/backup-glpi.log` et vérifiez la présence du nouveau fichier dans `/backups/glpi/`. Une sauvegarde automatique qu'on ne vérifie jamais est presque aussi dangereuse que pas de sauvegarde du tout.

Préparation — Création de données de test dans GLPI

Avant de simuler une perte, je m'assure d'avoir des tickets dans GLPI. Je me connecte à l'interface web de GLPI et je crée les 5 tickets suivants via **Assistance** → **Créer un ticket** :

The screenshot shows the GLPI web interface. At the top, there's a navigation bar with 'Accueil', 'Assistance', and 'Tickets'. Below it, a summary bar shows 5 tickets in various states: Tickets (5), Tickets entrants (0), Tickets en attente (0), Tickets assignés (5), Tickets planifiés (0), Tickets résolus (0), and Tickets fermés (0). The main table lists 5 tickets with columns: ID, TITRE, STATUT, DERNIÈRE MODIFICATION, DATE D'OUVERTURE, PRIORITÉ, DEMANDEUR, ATTRIBUÉ À, and CATÉGORIE. The tickets are:

ID	TITRE	STATUT	DERNIÈRE MODIFICATION	DATE D'OUVERTURE	PRIORITÉ	DEMANDEUR	ATTRIBUÉ À	CATÉGORIE
5	Accès Wi-Fi impossible - Ligue de basketball	En cours (Attribué)	2026-02-17 11:20	2026-02-17 11:20	Majeure	glpi	glpi	Réseau
4	Mise à jour du navigateur sur les postes de la salle de réunion	En cours (Attribué)	2026-02-17 11:18	2026-02-17 11:18	Basse	glpi	glpi	Logiciel
3	Écran bleu récurrent sur poste accueil	En cours (Attribué)	2026-02-17 11:17	2026-02-17 11:17	Haute	glpi	glpi	Matériel
2	Demande de création de compte - Ligue de tennis	En cours (Attribué)	2026-02-17 11:16	2026-02-17 11:16	Moyenne	glpi	glpi	Réseau
1	Imprimante réseau HS - Ligue de football	En cours (Attribué)	2026-02-17 11:15	2026-02-17 11:15	Haute	glpi	glpi	Matériel

Ces tickets représentent des demandes réalistes que la M2L pourrait recevoir de ses ligues sportives. Une fois créés, je relance une sauvegarde manuelle pour être sûr qu'ils sont inclus dans le dump :

Commande : `/usr/local/bin/backup-glpi.sh`

```
root@srv-glpi:~# /usr/local/bin/backup-glpi.sh
[OK] Sauvegarde réussie : glpi_20260217-122222.sql.gz
[OK] Rotation effectuée (conservation : 7 jours)
```

Étape 1 — Simuler une perte de données

On va supprimer une table importante de la base GLPI pour simuler une corruption. Je me connecte à MariaDB et supprime la table des tickets : Commande : `mysql -u glpi -p glpi`

```
root@srv-glpi:~# mysql -u glpi -p glpi
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 365
Server version: 10.11.14-MariaDB-0+deb12u2 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

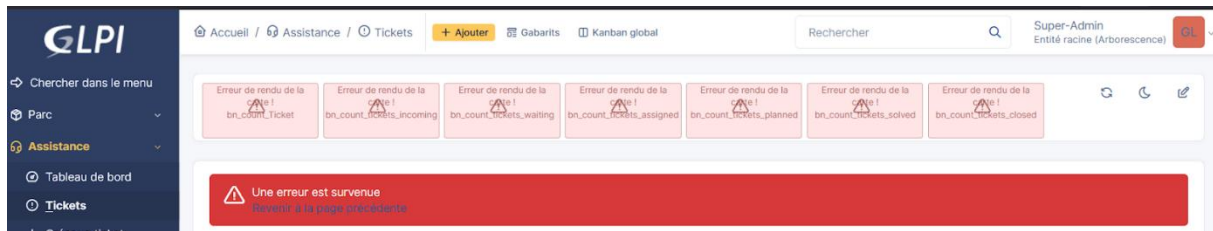
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [glpi]> SELECT COUNT(*) FROM glpi_tickets;
+-----+
| COUNT(*) |
+-----+
| 5 |
+-----+
1 row in set (0,001 sec)

MariaDB [glpi]> DROP TABLE glpi_tickets;
Query OK, 0 rows affected (0,016 sec)

MariaDB [glpi]> EXIT;
Bye
root@srv-glpi:~#
```


Lorsque j'accède à l'interface web de GLPI et que j'ouvre la page des tickets, une erreur SQL apparaît. Cela indique que la base de données est corrompue et qu'il est nécessaire de la restaurer.



Étape 2 — Identification de la sauvegarde à restaurer

Je liste les sauvegardes disponibles et je choisis la plus récente avec la commande suivante :

```
ls -lht /backups/glpi/
```

```
root@srv-glpi:~# ls -lht /backups/glpi/
total 2,4M
-rw-r--r-- 1 root root 224K 17 févr. 12:22 glpi_20260217-122222.sql.gz
-rw-r--r-- 1 root root 223K 17 févr. 11:49 glpi_20260217-114932.sql.gz
-rw-r--r-- 1 root root 2,0M 17 févr. 11:16 glpi_20260217-111651.sql
```

L'option -t trie par date de modification (la plus récente en premier). On utilisera `glpi_20260217-122222.sql.gz`.

Étape 3 — Restauration depuis le dump compressé

La restauration se fait en décompressant le fichier et en l'injectant directement dans MariaDB. L'opérateur pipe évite de créer un fichier intermédiaire décompressé :

```
root@srv-glpi:~# gunzip -c /backups/glpi/glpi_20260217-122222.sql.gz | mysql glpi
```

Détail de la commande :

Élément	Rôle
---------	------

<code>gunzip -c</code>	Décompresse vers la sortie standard (sans supprimer le fichier .gz d'origine)
------------------------	---

<code> </code>	Envoie le flux SQL décompressé directement à la commande suivante
----------------	---

<code>mysql glpi</code>	Exécute les instructions SQL dans la base glpi (authentification via /root/.my.cnf)
-------------------------	---

Étape 4 — Vérifier la restauration

Je vérifie que la table supprimée est de retour et que les données sont cohérentes :

```
root@srv-glpi:~# mysql -u glpi -p glpi -e "SELECT COUNT(*) FROM glpi_tickets;
```

Les 5 tickets sont de retour. Je recharge la page GLPI dans le navigateur : l'application fonctionne à nouveau normalement, sans aucune erreur SQL.

```

root@srv-glpi:~# ls -lht /backups/glpi/
total 2,4M
-rw-r--r-- 1 root root 224K 17 févr. 12:22 glpi_20260217-122222.sql.gz
-rw-r--r-- 1 root root 223K 17 févr. 11:49 glpi_20260217-114932.sql.gz
-rw-r--r-- 1 root root 2,0M 17 févr. 11:16 glpi_20260217-111651.sql
root@srv-glpi:~# gunzip -c /backups/glpi/glpi_20260217-122222.sql.gz | mysql glpi
root@srv-glpi:~# mysql -u glpi -p glpi -e "SELECT COUNT(*) FROM glpi_tickets;"
Enter password:
+-----+
| COUNT(*) |
+-----+
| 5         |
+-----+

```

Accueil / Assistance / Tickets + Ajouter Gabarits Kanban global Rechercher Super-Admin Entité racine (Arborescence) GL

5 Tickets	0 Tickets entrants	0 Tickets en attente	5 Tickets assignés	0 Tickets planifiés	0 Tickets résolus	0 Tickets fermés
-----------	--------------------	----------------------	--------------------	---------------------	-------------------	------------------

Filtrer par Vue Trié par Dernière modification

ID	TITRE	STATUT	DERNIÈRE MODIFICATION	DATE D'OUVERTURE	PRIORITÉ	DEMANDEUR - DEMANDEUR	ATTRIBUÉ À - TECHNICIEN	CATÉGORIE	TTR
5	Accès Wi-Fi impossible - Ligue de basketball	En cours (Attribué)	2026-02-17 11:20	2026-02-17 11:20	Majeure	glpi i	glpi i	Réseau	
4	Mise à jour du navigateur sur les postes de la salle de réunion	En cours (Attribué)	2026-02-17 11:18	2026-02-17 11:18	Basse	glpi i	glpi i	Logiciel	
3	Écran bleu récurrent sur poste accueil	En cours (Attribué)	2026-02-17 11:17	2026-02-17 11:17	Haute	glpi i	glpi i	Matériel	
2	Demande de création de compte - Ligue de tennis	En cours (Attribué)	2026-02-17 11:16	2026-02-17 11:16	Moyenne	glpi i	glpi i	Réseau	
1	Imprimante réseau HS - Ligue de football	En cours (Attribué)	2026-02-17 11:15	2026-02-17 11:15	Haute	glpi i	glpi i	Matériel	

20 lignes / pages De 1 à 5 sur 5 lignes

Vérifications finales

- Capture 1 : lsblk + lancement du script + ls -lh /backups/glpi/

```

root@srv-glpi:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0    0   60G  0 disk
├─sda1                              8:1    0  487M  0 part /boot
├─sda2                              8:2    0    1K  0 part
├─sda5                              8:5    0  59,5G  0 part
│   ├─deb12--master--vg-root        254:0    0  58,6G  0 lvm /
│   └─deb12--master--vg-swap_1      254:1    0   976M  0 lvm [SWAP]
sdb                                  8:16    0   20G  0 disk
└─sdb1                              8:17    0   20G  0 part /backups
sr0                                  11:0    1  738M  0 rom
root@srv-glpi:~# /usr/local/bin/backup-glpi.sh
[OK] Sauvegarde réussie : glpi_20260502-142531.sql.gz
[OK] Rotation effectuée (conservation : 7 jours)
root@srv-glpi:~# ls -lh /backups/glpi/
total 3,1M
-rw-r--r-- 1 root root 2,0M 17 févr. 11:16 glpi_20260217-111651.sql
-rw-r--r-- 1 root root 224K 2 mai 14:00 glpi_20260502-140011.sql.gz
-rw-r--r-- 1 root root 224K 2 mai 14:09 glpi_20260502-140906.sql.gz
-rw-r--r-- 1 root root 224K 2 mai 14:24 glpi_20260502-142418.sql.gz
-rw-r--r-- 1 root root 224K 2 mai 14:24 glpi_20260502-142436.sql.gz
-rw-r--r-- 1 root root 224K 2 mai 14:25 glpi_20260502-142531.sql.gz
root@srv-glpi:~# crontab -l_

```

- Capture 2 : crontab -l

```
root@srv-glpi:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# Sauvegarde quotidienne de la base GLPI - M2L
0 2 * * * /usr/local/bin/backup-glpi.sh >> /var/log/backup-glpi.log 2>&1
root@srv-glpi:~#
```

- Capture 3 : journalctl -u cron --no-pager

```
févr. 17 10:39:01 srv-glpi CRON[1957]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 10:39:01 srv-glpi CRON[1958]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
févr. 17 10:39:01 srv-glpi CRON[1957]: pam_unix(cron:session): session closed for user root
févr. 17 11:09:02 srv-glpi CRON[1955]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 11:09:02 srv-glpi CRON[1956]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
févr. 17 11:09:02 srv-glpi CRON[1955]: pam_unix(cron:session): session closed for user root
févr. 17 11:17:01 srv-glpi CRON[1917]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 11:17:01 srv-glpi CRON[1918]: (root) CMD ( cd / && run-parts --report /etc/cron.hourly)
févr. 17 11:17:01 srv-glpi CRON[1917]: pam_unix(cron:session): session closed for user root
févr. 17 11:39:01 srv-glpi CRON[1915]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 11:39:01 srv-glpi CRON[1916]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
févr. 17 11:39:01 srv-glpi CRON[1915]: pam_unix(cron:session): session closed for user root
févr. 17 12:09:01 srv-glpi CRON[1903]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 12:09:01 srv-glpi CRON[1904]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
févr. 17 12:09:01 srv-glpi CRON[1903]: pam_unix(cron:session): session closed for user root
févr. 17 12:17:01 srv-glpi CRON[1730]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 12:17:01 srv-glpi CRON[1730]: pam_unix(cron:session): session closed for user root
févr. 17 12:39:01 srv-glpi CRON[1815]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 12:39:01 srv-glpi CRON[1816]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
févr. 17 13:09:01 srv-glpi CRON[1902]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 13:09:01 srv-glpi CRON[1902]: pam_unix(cron:session): session closed for user root
févr. 17 13:17:01 srv-glpi CRON[1961]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
févr. 17 13:17:01 srv-glpi CRON[1962]: (root) CMD ( cd / && run-parts --report /etc/cron.hourly)
févr. 17 13:17:01 srv-glpi CRON[1961]: pam_unix(cron:session): session closed for user root
-- Boot 1d938154dc94df83dc5c458a12ff68 --
mars 08 20:37:52 srv-glpi systemd[1]: Started cron.service - Regular background program processing daemon.
mars 08 20:37:52 srv-glpi cron[667]: (CRON) INFO (pidfile fd = 3)
mars 08 20:37:52 srv-glpi cron[667]: (CRON) INFO (Running @reboot jobs)
mars 08 20:39:01 srv-glpi CRON[1175]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
mars 08 20:39:01 srv-glpi CRON[1176]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
mars 08 20:39:01 srv-glpi CRON[1175]: pam_unix(cron:session): session closed for user root
-- Boot 0edfb81931b64e59826eb0940c60eaf8 --
avril 23 14:25:02 srv-glpi systemd[1]: Started cron.service - Regular background program processing daemon.
avril 23 14:25:02 srv-glpi cron[666]: (CRON) INFO (pidfile fd = 3)
avril 23 14:25:02 srv-glpi cron[666]: (CRON) INFO (Running @reboot jobs)
-- Boot b2399eacbb954265b3f48172ca2abd81 --
avril 23 14:34:53 srv-glpi systemd[1]: Started cron.service - Regular background program processing daemon.
avril 23 14:34:53 srv-glpi cron[650]: (CRON) INFO (pidfile fd = 3)
avril 23 14:34:53 srv-glpi cron[650]: (CRON) INFO (Running @reboot jobs)
-- Boot ddb2e39cae0543c591acf226ad719745 --
mai 02 13:48:35 srv-glpi systemd[1]: Started cron.service - Regular background program processing daemon.
mai 02 13:48:35 srv-glpi cron[671]: (CRON) INFO (pidfile fd = 3)
mai 02 13:48:35 srv-glpi cron[671]: (CRON) INFO (Running @reboot jobs)
mai 02 14:09:45 srv-glpi CRON[1287]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
mai 02 14:09:45 srv-glpi CRON[1288]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
mai 02 14:09:45 srv-glpi CRON[1287]: pam_unix(cron:session): session closed for user root
mai 02 14:17:01 srv-glpi CRON[1349]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
mai 02 14:17:01 srv-glpi CRON[1350]: (root) CMD ( cd / && run-parts --report /etc/cron.hourly)
mai 02 14:17:01 srv-glpi CRON[1349]: pam_unix(cron:session): session closed for user root
root@srv-glpi:~# journalctl -u cron --no-pager
```

Commande : journalctl -u cron --no-pager

Ces captures présentent la validation de la solution de sauvegarde mise en place. La première montre la détection du disque de sauvegarde, l'exécution du script et la présence des fichiers .sql.gz dans /backups/glpi/. La deuxième confirme la planification de la tâche avec crontab -l, tandis que la troisième prouve, via journalctl, que le service cron fonctionne correctement et prend bien en compte les tâches programmées.